AGILE SOFTWARE ENGINEERING SKILLS

LECTURE 05

Agile Ceremonies

2024 / 2025

Springer
Julian Michael Bass

Outline

1. Introduction
2. Iteration Planning
3. Coordination Meetings
4. Customer Demonstrations
5. Pair Programming
6. Test–Driven Development
7. Specialist Agile Ceremonies

Introduction

- Ceremonies are the group collaboration activities
  - performed as part of an agile development process;
- Ceremonies are usually meetings, of one sort or another,
  - conducted during each iteration

Ceremonies types and Its benefits

- Agile ceremonies are structured meetings that guide the rhythm of Agile teams.
- They promote:
  - Transparency
  - Collaboration
  - Continuous delivery
- Core ceremonies include:
  - Iteration Planning
  - Daily Stand-ups
  - Customer Demonstrations
  - Retrospectives

Iteration Planning 1 to 13

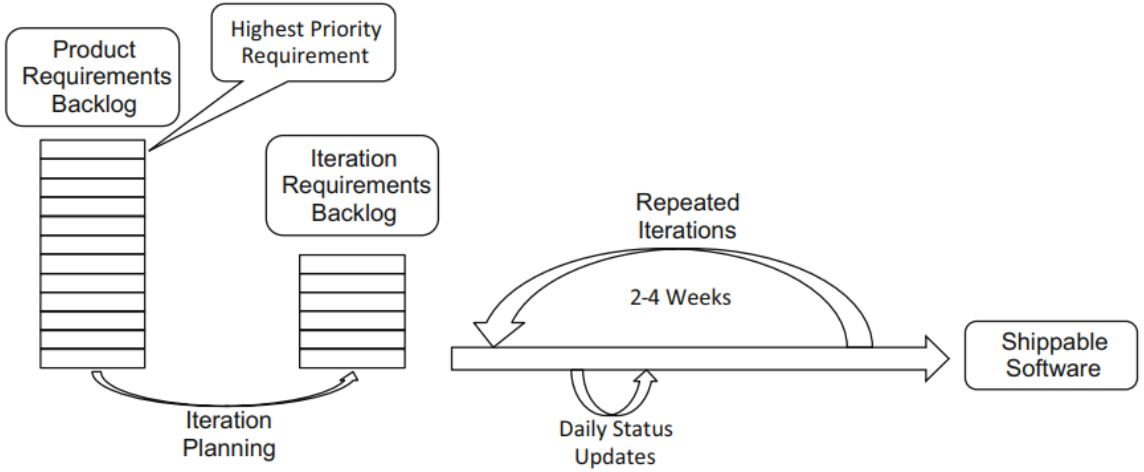
- Planning is essentially about deciding what to work on (and consequently what not to work on) in the coming (hopefully short) time window.
- We want to plan for a short iteration, as a way of **mitigating** the **risk** of **change**:
  - in the environment.
  - in customer needs or wishes.
  - in the teams and so on.
- Planning involves **mapping** customer priorities to estimates of our production capacity.

Iteration Planning 2 to 13

- Iteration planning is conducted at the start of each iteration, and comprises four tasks:
  1. prioritization of requirements,
  2. breaking up of requirements into technical tasks,
  3. estimation of technical tasks and consequently requirements,
  4. work item assignment within the team.

a technical task is a document that lists a **set of rules** and **constraints** that the future program must adhere to.

Iteration Planning 3 to 13



- On large–scale projects, more extensive planning is required.
- Release plans are used to coordinate cooperating teams and periodic risk assessments performed to prepare mitigations;
- let's focus on merely planning for the iteration ahead.

Iteration Planning 4 to 14 (Prioritization)

- **Prioritization**
- The product owner prioritizes **requirements** for implementation.
  - There is one exception, which is where team members notice some technical dependency between **tasks**.
  - That is, some high–priority requirement, selected by the product owner, depends on some lower–priority requirement being **implemented first**.
  - In this situation, the team can advise the product owner that the higher–priority requirement can be implemented but will not work.
  - The product owner can then decide if they want to increase the **priority** of the lower–priority requirement.

Iteration Planning 5 to 13 (Features and Technical Tasks)

- **Features and Technical Tasks**
  - Our first task during iteration planning, with a prioritized requirements backlog, is to get consensus on what a user story comprises.
  - We break each feature up into a set of technical tasks.
    - Does this user story require any front-end interface screens?
    - Does this feature need to use data storage?
    - What business logic operations are part of this feature?
  - We need to divide the user story into all its constituent technical tasks.
  - By creating a list of smaller work items, we can more confidently estimate the effort required to implement each and hence the overall feature.
    - think about user interface tasks,
    - application logic and data storage tasks as separate items.
  - Repeat the process for the next high-priority user story in the product backlog.

Iteration Planning 6 to 13 (Estimation)

- **Estimation**
  - We need to know how many features we can fit into an **iteration**.
    - That is a difficult question to answer.
    - Not least because team members may have different perceptions (notices) of what is required to implement a feature.
  - The two techniques worth mentioning are
    - story points.
    - T–shirt sizing.
  - Both approaches tend to use the planning poker technique.

Iteration Planning 7 to 13 (Estimation)

- **Story Point Estimation**
- Story points are a relative measure of the **size or complexity** of user stories.
- The integers used to approximate size are taken from a Fibonacci number sequence: 1, 2, 3, 5, 8 and 13.
- Using this number sequence, the estimates for larger sizes are less precise; consequently, there is no need to differentiate between sizes 9 and 10.
  - Instead, it is sufficient to distinguish between 8 and 13.
- Larger, story point sizes, depending upon the business domain of the application under development, could indicate that the user story is in fact an epic that needs to be further decomposed into user stories.
- If large user stories cannot logically be decomposed, then story point sizes like 20, 40 or 100 might be considered.
- However, we do need each user story to fit into an iteration; consequently, epics do have an upper size limit (or iteration durations lengthened).

Iteration Planning 8 to 13 (Estimation)

Planning poker is commonly used to allocate story points to technical tasks. To perform planning poker, the team members collectively:

1. Take each technical task; in turn, the first round of voting starts,
2. Discuss each technical task, if necessary,
3. Write down (secretly) their estimates for the work item,
4. When everyone has finished writing, team members reveal their votes for the tasks,
5. Look at the story points assigned and see if there is close consensus (in novice teams or a new application domain, close consensus is unlikely),
6. If there is consensus, on the story point allocation, move on to the next technical task,

Iteration Planning 9 to 13 (Estimation)

7. If there is no consensus, constructively discuss the highest and lowest story point estimates and try to understand why someone thought it was a larger or smaller task,
8. Following this discussion, move into a second round of voting
9. Continue rounds of voting and discussion until consensus emerges around the story point value for a task,
10. Then, move on to the next technical task or user story.

- The planning poker approach to estimation is consensus–based and draws upon all the team expertise available.
- This approach fosters discussion, which is a valuable source of learning for novice or less experienced members.
- Teams tend to improve estimation accuracy over time

Iteration Planning 10 to 13 (Estimation)

- **T–Shirt Sizing**
  - A simple and easy way to estimate tasks is to agree a small set of categories and fit the features into the agreed groups.
  - So, we can think of tasks or features as being small, medium, large and extra large.
  - We think of this estimation process as fitting tasks in a (small) group of size categories.
    - How many size categories are reasonable for your context?
    - Three, four or five?
    - How accurate (or precise) do you expect your estimation to be?
  - More than five size categories require considerable effort for a novice team.
  - If a task is bigger than the usual range of categories, we take further action, as we did with story point estimation.
  - For example, if a task is extra, extra large (which is an epic user story), it requires further analysis to break it down into a more manageable size.

Iteration Planning 11 to 13 (Estimation)

- We can use a similar planning poker process as we used with story point estimation.
- Everyone secretly writes their size estimate for a technical task on a sticky note. The sticky notes are all revealed simultaneously (so no one can change their score, when they see other people's estimates).
- Team members can then see the variation in size estimates within the group.
- Teams usually discuss the thinking behind the largest and smallest estimates.
- After the discussion, team members are invited to offer a revised size estimate. Everyone writes a second estimate on a sticky note, which is then shared again. After a few rounds of discussion and voting,
- consensus is achieved on the size of that specific technical task.

Iteration Planning 12 to 13 (Estimation)

T-Shirt Sizing, Burndown Charts and Velocity

Some people describe agile methods, like Scrum, as *empirical* methods. The idea is we can keep track of the number of story points completed in an iteration. This figure, the number of story points completed in an iteration, is called the team *velocity*. We can also plot burndown charts, during the iteration, showing story points as they are completed

A perceived disadvantage of T-shirt sizing is the lack of integer values for each size. A simple way to combine velocity, burndown chart and T-shirt sizing is to consistently assign an integer value for each size. For example, large might equate to 13 points. Medium equals 8 points and so on. Now we can use T-shirt sizing as an empirical method.

Iteration Planning 13 to 13 (Task Assignment)

- **Task Assignment**
- After requirements have been estimated, we now have an idea how many tasks can fit into an iteration.
- We can now decide **who**, in the team, is going to tackle each task.
- A defining characteristic of a self-organizing team is that people volunteer for tasks.
  - Sometimes, people pick up a task because it is similar to others they have successfully completed in the past.
  - On the other hand, sometimes people pick up tasks to learn something new.
- The aspiration for the team, achieved in experienced groups, is that anyone be capable of doing any tasks.
- You might imagine relying on volunteers to pick up tasks means that there are tasks no one wants that don't get taken up. But practitioners say this is unusual.
- More commonly, self-organizing teams develop a sense of shared commitment to group outcomes.
- So, unpopular tasks do tend to get shared around the group over **successive iterations**.

Relationship in terms of terminology

- Features often become Epics
- Epics are broken into User Stories
- User Stories are implemented via Tasks
- Epic is a big-picture functionality that spans multiple sprint
- A feature and a requirement often overlap
  - A feature is a capability that users care about.
  - A requirement is a condition that must be met to support that capability.
  - Features are often captured as user stories in the product backlog
  - Technical tasks are the implementation details needed to realize a feature.
- Epic: "Enable users to manage their accounts"
- User Stories under that epic:
- "As a user, I want to update my profile info"
- "As a user, I want to change my password"
- "As a user, I want to delete my account"
- This hierarchy helps teams plan, prioritize, and deliver value incrementally.

Coordination Meetings 1 to 8

- The daily stand-up, a coordination meeting involving everyone in the team, is an important activity in agile methods. It is where **everyone** finds out what is going on in the team.
- Everyone in the team answers the following three questions:
  1. What have I been doing, since the last stand-up?
  2. What will I be doing, between now and the next stand-up?
  3. Are there any impediments preventing me from making progress?

Coordination Meetings 2 to 8

- Some groups like to add a fourth question:
- Am I going to create any blockers that might impede others?
- This fourth question is typically useful in larger projects where there are dependencies between the codes produced by different team members.

Example: Why Might Anyone Create an Impediment?
The fourth question *am I going to create any blockers that might impede others?* is sometimes useful where one or more team members are creating a class or API which is relied upon by other members of the team. When changes are made to that API, this could potentially cause code already written by others to fail. In such cases, it is polite to warn people that the interface or API they use is changing.

Coordination Meetings (Virtual Stand-Up Meetings) 3 to 8

- **Virtual Stand-Up Meetings**
- Where groups are working remotely or geographically distributed, a stand-up meeting can be conducted online.
- Obviously, given the choice, we'd all rather be in the same room at the same time.
- However, there are often lots of reasons why someone isn't able to physically be with the rest of the team.
- The ubiquity of video or audio conferencing facilities makes virtual team meetings more attractive than not having a meeting at all.

Coordination Meetings (Virtual Stand–Up Meetings) 4 to 8

- Partly online coordination meetings are common, that is, where one or two members of the team participate online, while the rest of the group gather around a Kanban board.
- If you are technical team member, you just need to answer your three questions when your turn comes, so that can work.
- But if you are the product owner, you are a kind of observer anyway, so that does not matter so much.

Coordination Meetings (Virtual Stand-Up Meetings) 5 to 8

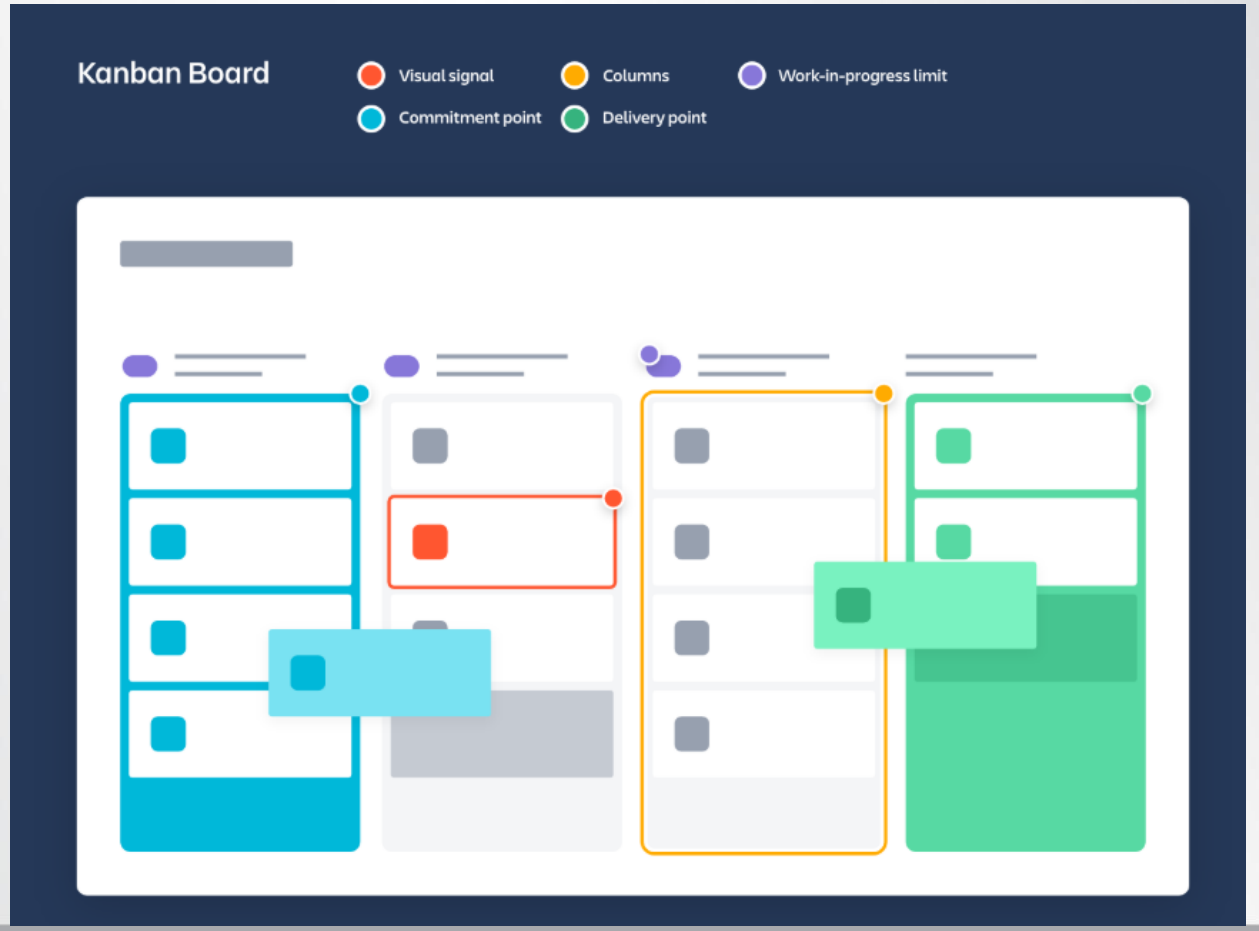
- However, being the only online participant in a meeting where everyone else is co-located is not fabulous (wonderful).
- The solitary (single) remote worker tends to feel left out of the discussion and often can't interject to make a comment.
- It is difficult to achieve a sense of team cohesion in this arrangement, trust is weak, and the risk of conflicts emerging is high

Coordination Meetings (Kanban) 6 to 8

- **Kanban**
- Coordination meetings are usually held in front of a visual (often physical) display of project status. The idea is to make visual the team's efforts towards project goals.

- The Kanban board originates in the world of advanced manufacturing and just in–time production emanating (emerging) from the Japanese car industry.
- In its simplest form, it consists of three columns: To Do, Doing and Done.
- The requirements, features or, more likely, technical tasks identified in iteration planning are added to the **board** using sticky notes.
- Each sticky note represents a technical task.
- All the tasks start off in the To Do **column**.

Coordination Meetings (Kanban) 7 to 8



Coordination Meetings (Kanban) 8 to 8

- As the project progresses, the sticky notes all work their way over to the ***Done*** column.
- The sticky notes are usually moved during coordination meetings, as the status of an item changes.
- This gives a visually appealing sense of project progress.
  - Each team member can see their effort as part of the wider range of team activities.
- Online tools, such as **Trello** , can be used to support virtual teams using Kanban boards.
- This retains the visual illustration of project progress while enabling **remote working**.
- Tasks or user stories modelled using online Kanban boards can be embellished (decorated) with acceptance test criteria and links to definitions of done.

Customer Demonstrations I to 5

- A customer demonstration is where you demonstrate working code to your customer or client at the **end of an iteration.**
- Preparation for the customer demonstration requires the following steps:
  - Scrum master arranges a convenient date and time with the product owner or client,
  - Scrum master arranges a venue (place), which might be online of course,
  - Scrum master makes sure that all the team members are available and aware of the time and venue,
  - Scrum master makes sure you have the right technology to demonstrate the software (install and check the demonstration environment),
    - Identify a team member to make notes during the customer demonstration,

Customer Demonstrations 2 to 5

- As a team, rehearse the demonstration, and make sure the demonstration runs smoothly and that any hand–off from one presenter to another is seamless,
- As a team, make sure you wear appropriate (usually smart casual or business) dress.
- During the customer demonstration, the meeting agenda is as follows:
  1. Introduce the purpose of the meeting.
  2. Review the requirements you were supposed to implement.
  3. Demonstrate each new feature of the software.
  4. Review any requirements that you were unable to implement for any reason.
  5. Collect and carefully record any feedback from the product owner or client.

Customer Demonstrations 3 to 5

- **An important benefit of the incremental development approach is:**
  - the idea of getting feedback at intermediate stage of the project development process.
- Customers, clients or users need to be able to see progress towards project completion and influence the direction of travel.
- You want reassurance that the code you are writing is fit for purpose and meets the needs that have been identified.
- You get reassurance from the customers that you are on the right track.
- Customers see evidence of progress towards the completed system.
- Demonstrate how each feature works and the defensive programming measures you have implemented.
  - So, for example, where your software requires user input, you will show how the programm responds if the wrong type of information is provided.
  - You may also demonstrate how the working software has benefited from testing and other quality assurance measures.

Customer Demonstrations 4 to 5 (Retrospectives)

- Another important benefit of the incremental development approach is the idea of learning from each iteration.
- Committed software developers genuinely (truthfully) want to improve their team effectiveness.
  - The retrospective is an opportunity to do that.
- Retrospectives are better than infrequent and ineffective end-of-project reviews.

Customer Demonstrations 5 to 5 (Retrospectives)

- A simple way you can conduct a retrospective is for everyone in the team to think of:
  1. Three things that worked well, in the previous iteration,
  2. Three things that, as a team, you could be doing better,
  3. Three improvement actions for the next sprint.
- As a result:
- The things that worked well are the good practices you want to keep doing.
- The things you could be doing better are the areas for potential improvement.
- As a team, you look for consensus areas.
  - Often you find several team members will point out similar areas where things could improve.
- Once you have identified three potential areas of improvement, from among the suggestions from team members you can develop a set of actions.
  - Actions are practical steps you can take to address improvement areas.

Pair Programming 1 to 2

- Pair programming is where two developers work together in a pilot–co–pilot configuration.
- This is not a case of one person programming while the other rests or watches.
  - Rather, it is that both developers are occupied on different activities during the development process.
- In pair programming, one developer is more focused on low–level syntax and language mechanics.
  - Usually, this developer has the keyboard and is actually typing source code.
- While one developer is typing and thinking about syntax, the other is considering higher–level structure and readability.
  - This second developer is focused on source code quality and acceptance testing.

Pair Programming 2 to 2

- **Another common use of pair programming is for developing new talent.**
- There is some evidence that novice team members operate at the level of the more experienced team member when working in pairs and that pairs significantly outperform individuals.
- Pair programming can be used to support new developers acquiring software skills for the first time.
- pair programming support the induction (training) of experienced software developers joining a team and learning to find their way around an existing code base. In either situation, the learner controls the keyboard.
- You don't learn much by merely watching an experienced hand.
- Obviously, the mentor adopts a warm, constructive and supportive demeanor (behavior).

Test-Driven Development 1 to 2

- Test-driven development takes a counter-intuitive approach to software development in which automated tests are written before the code itself.
  - Developers extract the test criteria (constraints) recorded against each requirement.
  - These test criteria are then written into unit tests.
  - The tests fail at first, when executed, because no code has been written.
  - Then code is written to meet the requirements.
  - One by one, the automated tests will pass.
  - At the end of the process, **code** will have been written to pass all the tests.
- So the general test-driven development cycle goes as follows:

  1. Write a test, 2. Make it run 3. Make it right.

Test-Driven Development 2 to 2

- Make it run means quickly filling in functionality to get tests to pass.
- Make it right means refactor the code into an elegant form, by removing duplication and simplifying, while still passing all the tests, of course.
- The approach has been shown experimentally to be less effective, in terms of defect reduction, than code inspections.
- But, it is suggested test-driven development improves developer morale, since the conclusion of the development process is signified by passed tests.

Specialist Agile Ceremonies 1 to 7

- There are some specialist ceremonies that development teams use, usually when things are not going well.
- Sometimes our initial estimates of a user story turn out to be wrong.
- Perhaps, as a development team, we misunderstood the requirement.
- Or, maybe implementation of the story turns out to be much more complicated than expected.
- Often, pair programming is enough to get us out of such a fix.
- However occasionally, a more dramatic solution is called for.

Specialist Agile Ceremonies 2 to 7

1. **Spikes: Time-boxed research tasks**
2. **Swarm Programming: Multiple devs tackle one issue**
3. **Mob Programming: Entire team works on one task together**

- **Used for:**
  - Complex problems
  - Urgent bugs
  - Learning sessions

Specialist Agile Ceremonies 4 to 7 (Spikes)

- A spike is where the estimate, for a requirement or technical task under development, proves to be inaccurate.
- This usually means some new, previous complexity associated with a requirement has emerged.
- We can mark the task on our Kanban board, re-estimate the effort required and re-prioritize.
- The advantage of treating the story as a spike is that we can remain committed to other stories in the sprint and do not get distracted with the troublesome (problem) one.
- We might choose to park the story for a future sprint.
- But, this is undesirable, because we have failed to meet our commitment to the product owner or client.
- In consultation with our product owner, we might decide that the spike is too important to just park for a future sprint.

Specialist Agile Ceremonies 5 to 7 (Spikes)

- Having identified a story as a spike, we might also consider adding resources to that story.
- In this situation, quite a lot of teams use pair programming to resolve the issue.
- While, in the Extreme Programming method, it is recommended that developers use pair programming all the time, some practitioners prefer to use pair programming only under specific circumstances.
- A common, special case, use of pair programming is to address spikes.
- Alternatively, we might reduce the priority of some other **activity**, so that we can resolve the spike.
- There are some other approaches to resolving spikes, such as swarm programming or even pulling in additional specialist support from outside the team

Specialist Agile Ceremonies 6 to 7 (Swarm Programming)

- In swarm programming, more than two developers work together.
- This can be useful if team members want to work together to tackle some new task or technology that no one has used before or where progress for the whole team is blocked by one particular problem that needs to be solved.
- The idea is that the swarm makes development quicker, and comes up with higher–quality solutions, **than** an individual or pair.
- Usually swarm programming is used to tackle specific issues.
  - For example, some teams use swarm programming to address a high–priority spike.
  - Alternatively, a swarm might be used to **achieve consensus** on an architectural style.

Specialist Agile Ceremonies 7 to 7 (Mob Programming)

- Mob programming takes the ideas of pair and swarm programming to the extreme.
- In mob programming, the whole team works together all the time.
- The team is co-located, working together at one computer performing all requirements, design and development activities.
- So, in mob programming, the team has workshops for **defining stories**, **working** with customers and **designing**, **testing** and **deploying** software.

AGILE SOFTWARE ENGINEERING SKILLS

End of Lecture 05

Agile Ceremonies

NEXT

Lecture Agile Ceremonies (Exercises)